

Guia Completo de Git e GitHub para Iniciantes

O que é Git?

Git é um sistema de controle de versão.

Ele serve para:

- salvar versões do código
 - voltar para versões antigas
 - trabalhar em equipe
 - organizar alterações no projeto
-

O que é GitHub?

GitHub é uma plataforma online onde você armazena repositórios Git.

Ele permite:

- hospedar projetos
 - colaborar com outras pessoas
 - fazer backup do código
 - compartilhar projetos
-

Instalação do Git

Baixar Git

Acesse:

<https://git-scm.com/downloads>

Instale normalmente.

Configuração Inicial

Abra o terminal e configure:

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@gmail.com"
```

Verifique:

```
git config --list
```

Criando um Repositório no GitHub

Passo 1

Entre no GitHub: <https://github.com>

Passo 2

Clique em:

New Repository

Passo 3

Escolha:

- nome do repositório
 - público ou privado
-

Passo 4

Clique em:

Create Repository

Criando um Projeto Git Local

Entrar na pasta

```
cd nome-da-pasta
```

Inicializar Git

```
git init
```

Isso cria o repositório local.

Entendendo o Fluxo do Git

O fluxo básico é:

```
Arquivos -> add -> commit -> push
```

Adicionando Arquivos

Adicionar tudo

```
git add .
```

Adicionar arquivo específico

```
git add app.js
```

Criando Commits

Commit = salvar uma versão do projeto.

```
git commit -m "primeiro commit"
```

Conectando ao GitHub

HTTPS

```
git remote add origin https://github.com/usuario/repositorio.git
```

SSH

```
git remote add origin git@github.com:usuario/repositorio.git
```

Enviando Código para o GitHub

Definir branch principal

```
git branch -M main
```

Fazer push

```
git push -u origin main
```

Fluxo Diário de Trabalho

Sempre que alterar algo:

```
git add .  
git commit -m "descrição da alteração"  
git push
```

Verificando Alterações

Status do projeto

```
git status
```

Ver Histórico de Commits

Histórico resumido

```
git log --oneline
```

Histórico completo

```
git log
```

Clonando um Repositório

Para baixar um projeto existente:

```
git clone URL_DO_REPOSITORIO
```

Exemplo:

```
git clone git@github.com:usuario/projeto.git
```

O que são Branches?

Branches são ramificações do projeto.

Elas permitem:

- desenvolver funcionalidades separadamente
- evitar quebrar o código principal

- trabalhar em equipe
-

Branch Principal

Normalmente:

- main
- ou master

É a versão principal do projeto.

Criando uma Branch

```
git checkout -b nome-da-branch
```

Exemplo:

```
git checkout -b tela-login
```

Mudando de Branch

```
git checkout nome-da-branch
```

Ver Todas as Branches

```
git branch
```

Enviando Branch para o GitHub

```
git push -u origin nome-da-branch
```

Fazendo Merge

Merge junta branches.

Exemplo:

Voltar para main

```
git checkout main
```

Fazer merge

```
git merge tela-login
```

Pull Requests

Pull Request (PR) é uma solicitação para juntar alterações.

Fluxo comum: 1. criar branch 2. desenvolver 3. enviar branch 4. abrir Pull Request 5. revisar código 6. fazer merge

Atualizando Projeto

Baixar alterações do GitHub

```
git pull
```

Diferença Entre Pull e Push

Push

Envia alterações locais para o GitHub.

```
git push
```

Pull

Baixa alterações do GitHub.

```
git pull
```

Arquivo .gitignore

Serve para ignorar arquivos.

Exemplo:

```
node_modules/  
.env  
dist/
```

Boas Práticas

Faça commits pequenos

✓ Bom:

```
git commit -m "adiciona validação do login"
```

✗ Ruim:

```
git commit -m "update"
```

Nunca envie:

- senhas
 - arquivos .env
 - node_modules
 - chaves privadas
-

Comandos Mais Importantes

Inicializar

```
git init
```

Adicionar arquivos

```
git add .
```

Commit

```
git commit -m "mensagem"
```

Push

```
git push
```

Pull

```
git pull
```

Ver status

```
git status
```

Ver branches

```
git branch
```

Fluxo Profissional Recomendado

```
git pull
git checkout -b nova-feature

# desenvolvimento

git add .
git commit -m "implementa nova feature"
git push -u origin nova-feature
```

Depois:

- abrir Pull Request
- revisar
- fazer merge

Resumo Final

Git:

- controla versões

GitHub:

- hospeda repositórios

Commit:

- salva alterações localmente

Push:

- envia para GitHub

Pull:

- baixa alterações

Branch:

- linha separada de desenvolvimento

Merge:

- junta alterações
-

Próximos Assuntos para Estudar

Depois deste guia, estude:

- conflitos de merge
- rebase
- stash
- cherry-pick
- GitFlow
- CI/CD
- GitHub Actions

Esses temas são muito usados profissionalmente.